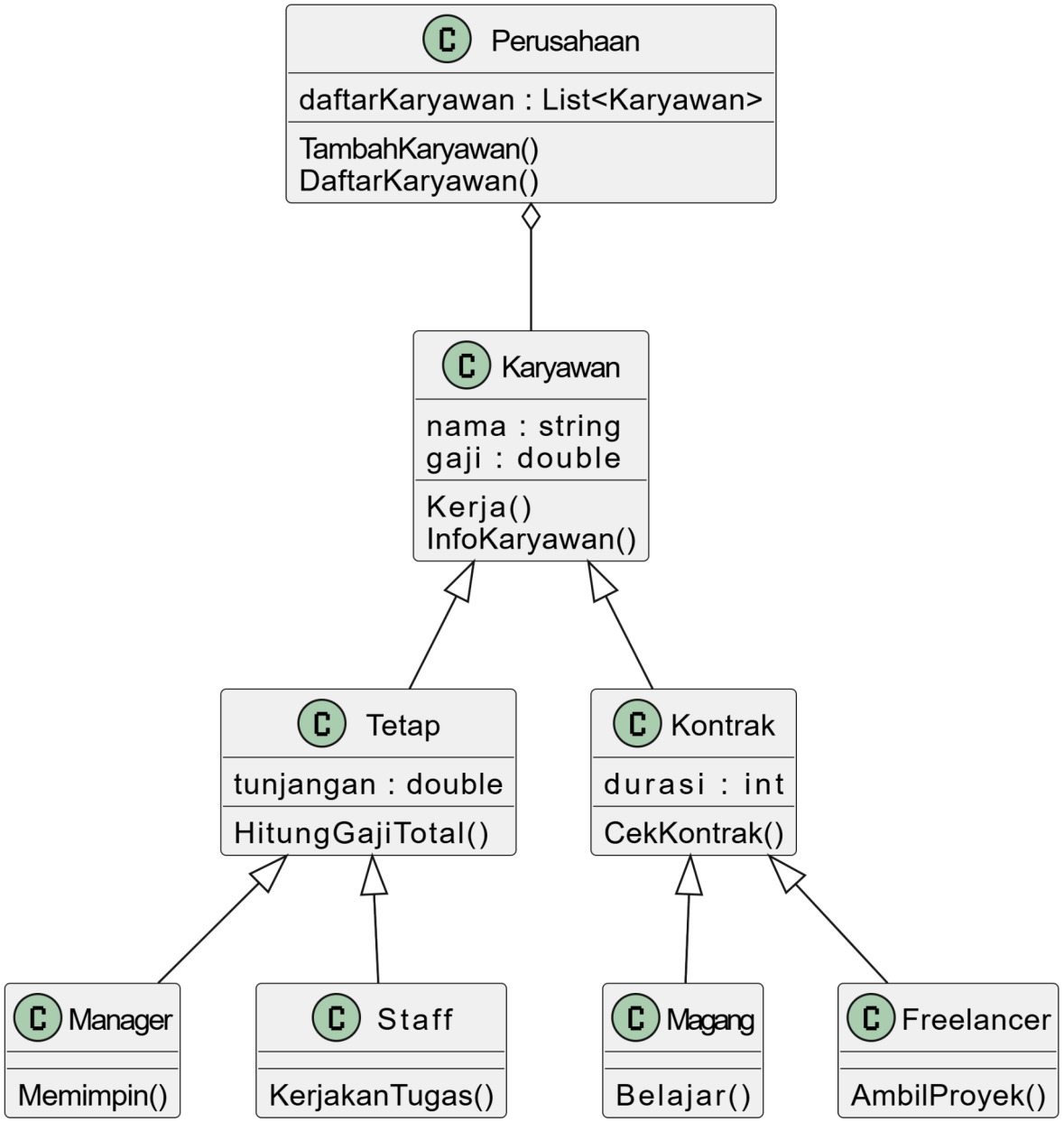
SOAL SISTEM KARYAWAN

Nama : Rayhanuun Aura Chantiqa
NIM : 252410103099
Kelas : C
Link Repository GitHub : https://github.com/vourcantika-source/tugas_praktikum_pbo

INSTRUKSI:

1. Buatlah sebuah project baru di Visual Studio, nama bebas.
2. Implementasikan kelas-kelas berikut:
    a. Kelas **Karyawan** dengan properti nama dan gaji
    b. Kelas **Tetap** yang mewarisi Karyawan, dengan properti tambahan tunjangan serta method HitungGajiTotal()
    c. Kelas **Kontrak** yang mewarisi Karyawan, dengan properti tambahan durasi serta method CekKontrak()
    d. Kelas **Manager** dan **Staff** yang mewarisi Tetap, masing-masing memiliki method Memimpin() dan KerjakanTugas()
    e. Kelas **Magang** dan **Freelancer** yang mewarisi Kontrak, masing-masing memiliki method Belajar() dan AmbilProyek()
    f. Kelas **Perusahaan** yang memiliki koleksi Karyawan
3. Implementasikan method pada kelas Karyawan:
    a. Kerja()
    b. InfoKaryawan()
4. Override method Kerja() di setiap subclass.
5. Dalam kelas Perusahaan, implementasikan:
    a. TambahKaryawan(Karyawan karyawan
    b. DaftarKaryawan()
6. Dalam method Main(), lakukan hal berikut:
    a. Buat objek perusahaan
    b. Buat beberapa objek karyawan
    c. Tambahkan ke perusahaan
    d. Tampilkan semua data
    e. Demonstrasikan polymorphism
    f. Panggil method khusus
7. Implementasikan kasus di setiap pertanyaan pada method Main, dan tuliskan jawaban Anda di kolom yang tersedia (semakin lengkap jawaban semakin baik).
8. Unggah project Anda ke Github dengan nama repository **KELAS_NIM_TASKPBO**, pastikan akses repository adalah **public**.
9. Simpan modul yang telah Anda kerjakan ke dalam format PDF.
10. Pengumpulan terakhir sesuai dengan ketentuan yang diberikan di kelas masing masing.
11. Link Pengumpulan : https://forms.gle/5KVcRH9JYByq7soc8


Perusahaan
daftarKaryawan : List<Karyawan>
TambahKaryawan()
DaftarKaryawan()
Karyawan
nama : string
gaji : double
Kerja()
InfoKaryawan()
Tetap
tunjangan : double
HitungGajiTotal()
Kontrak
durasi : int
CekKontrak()
Manager
Memimpin()
Staff
KerjakanTugas()
Magang
Belajar()
Freelancer
AmbilProyek()

1	Soal	Jalankan method Kerja() pada objek Manager dan Freelancer. Apa yang terjadi? Mengapa demikian?
	Jawaban	Hasilnya beda. Manager menampilkan "Budi (Manager) mengatur strategi dan mengawasi tim.", sedangkan Freelancer menampilkan "Rina (Freelancer) mengerjakan proyek secara mandiri." Hal ini terjadi karena method Kerja() di kelas Karyawan dibuat virtual, lalu di-override di masing-masing subclass. Jadi tiap kelas punya isi Kerja() sendiri, dan yang dijalankan adalah milik kelas objeknya.

2	Soal	Jalankan method Memimpin() pada objek Manager. Apa yang terjadi? Mengapa demikian?
	Jawaban	Method berjalan dan menampilkan "Budi sedang memimpin rapat tim." Karena Memimpin() memang dibuat di kelas Manager, dan objek yang memanggilnya juga bertipe Manager. Method ini hanya ada di Manager, kelas lain seperti Staff atau Freelancer tidak punya.

3	Soal	Dapatkah Anda menampilkan informasi lengkap (nama, gaji, tunjangan) dari objek Manager? Bagaimana caranya?
	Jawaban	Bisa, dengan memanggil manager.InfoKaryawan(). Method ini di-override di kelas Tetap: pertama memanggil base.InfoKaryawan() untuk menampilkan nama dan gaji, lalu menambah tunjangan dan gaji total. Manager mewarisinya dari Tetap. Hasilnya: Nama: Budi, Gaji: Rp15,000,000 lalu Tunjangan: Rp5,000,000, Gaji Total: Rp20,000,000. Bisa juga dengan memanggil properti satu-satu (manager.Nama, manager.Gaji, manager.Tunjangan), karena Manager mewarisi properti dari Tetap dan Karyawan.

4	Soal	Jalankan method Belajar() pada objek Magang. Apa yang terjadi? Mengapa demikian?
	Jawaban	Method berjalan dan menampilkan "Dani sedang belajar dari mentor." Karena Belajar() ada di kelas Magang dan objeknya bertipe Magang. Method ini khusus untuk Magang, jadi objek Freelancer tidak bisa memanggilnya (akan error).

5	Soal	Buatlah variabel bertipe Karyawan, lalu isi dengan objek Staff. Jalankan method Kerja(). Apa yang terjadi? Mengapa demikian?
	Jawaban	Kodenya: Karyawan k = new Staff("Andi", 7000000, 1500000); lalu k.Kerja();Yang tampil adalah Kerja() milik Staff, yaitu "Andi (Staff) mengerjakan tugas operasional.", bukan milik Karyawan. Ini disebut polymorphism. Variabel bertipe Karyawan boleh diisi objek Staff karena Staff adalah turunan Karyawan. Karena Kerja() di-override, yang dijalankan mengikuti objek aslinya (Staff). Tapi lewat variabel ini, method khusus Staff seperti KerjakanTugas() tidak bisa dipanggil kecuali di-casting dulu ((Staff)k).KerjakanTugas().

6	Soal	Kesimpulan apa yang dapat Anda ambil?
	Jawaban	Dari percobaan ini bisa disimpulkan bahwa inheritance membuat subclass otomatis punya properti dan method dari kelas induknya, jadi tidak perlu menulis ulang kode. Dengan virtual dan override, tiap subclass bisa punya cara kerja sendiri untuk method yang namanya sama. Lalu dengan

		polymorphism, kita bisa menyimpan berbagai jenis karyawan dalam satu tipe Karyawan (misalnya di List di kelas Perusahaan) dan memanggil Kerja() sekali untuk semuanya, hasilnya tetap sesuai jenis masing-masing. Method khusus seperti Memimpin() atau Belajar() hanya bisa dipakai oleh kelas yang memilikinya.